

Trabajo Práctico Integrador 2026

Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

Materia: Programación Funcional

Grupo: 42

Lenguaje asignado (Fase 3): OCaml

Fecha de entrega: 16 de junio de 2026

Integrantes	/	Usuario de github
-------------	---	-------------------

Otero Manuel	/	Manuotero11
Facundo Gaston Roda	/	Rodafacundogaston
Rodriguez Agustina Ailén	/	Agusrodriguez-hub
Joaquina Aymara Castañeda	/	Joacast
Ernesto Silguero	/	Erness266

1. Introducción y contexto

Las ciudades modernas requieren sistemas de tráfico inteligentes para optimizar el flujo vehicular y garantizar la seguridad vial. En este trabajo desarrollamos el núcleo lógico de un sistema embebido de control de semáforos para intersecciones críticas, implementado en Common Lisp aplicando estrictamente el paradigma funcional: funciones puras, inmutabilidad y composición. El trabajo se organiza en tres fases:

Fase 1: lógica de control (Requerimientos 1 a 6) en Common Lisp.

Fase 2: integración del ecosistema con el gestor de paquetes Quicklisp (librería local-time): la auditoría (Requerimiento 3) pasa a mostrar la fecha y hora en formato legible en lugar del epoch Unix.

Fase 3: re implementación de transición y timer en OCaml y análisis comparativo entre paradigmas.

2. Fase 1 - Diseño funcional adoptado

2.1. Principios aplicados

Todo el núcleo (lisp/core.lisp) respeta las normas establecidas en el documento de trabajo integrador.

1. Inmutabilidad absoluta. No se utiliza `defparameter/defvar` para guardar estados cambiantes, ni operadores destructivos (`setq`, `setf`). El estado del tráfico (el color) y el tiempo (el timestamp Unix) fluyen únicamente como argumentos de las funciones. La

configuración de tiempos se modela como una lista de asociación (alist) que se construye fresca en cada llamada a tiempos-por-defecto; nadie comparte ni puede mutar ese estado.

2. Cero bucles imperativos. No usamos loop, dolist, dotimes ni while.

3. Composición. Funciones pequeñas y puras se combinan para resolver problemas mayores: timer se apoya en duración-ciclo y color-en-posición; ciclos-por-tiempo y distribución-horaria reutilizan duración-ciclo.

2.2. Modelo de datos

Estados del semáforo: símbolos en-rojo, en-amarillo, en-verde.

Colores destino: símbolos rojo, amarillo, verde.

Configuración de tiempos: alist ((: rojo. 90) (: amarillo. 6) (: verde. 120)), construida fresca por la función pura tiempos-por-defecto (no es una variable global mutable). Al recibirse siempre por argumento, la fuente de los tiempos podría intercambiarse sin tocar el resto del código.

Secuencia temporal real: rojo -> verde -> amarillo -> (rojo). Es la base tanto del temporizador como de la tabla de transiciones válidas.

2.3 Requerimientos implementados

Requerimientos y Funciones

Qué se revuelve en cada uno resuelve:

1. Transición: Estado siguiente + acción "cambiar-a-color", pasa acción-por-defecto si la transición no es válida.
2. Timer y color-en-posición: Color activo en un timestamp Unix dado (ciclo de 216 s).
3. Auditar-cambio: Logging forense del cambio de estado.
4. Duración-ciclo, recomendación-ciclo: Duración total del ciclo y recomendación según la regla psicológica (35-150 s).
5. Ciclos-por-tiempo Cantidad de ciclos completos en N minutos.
6. Distribución-horaria: Porcentaje de tiempo de cada color en 1 hora.
7. Casos de prueba: Pruebas alternativas y errores.

2.4 Clasificación taxonómica de las funciones.

Cada función lleva un encabezado de clasificación.

Función	Naturaleza	Estrategia de control	Impacto en la memoria
tiempos-por-defecto	Pura	Constructora simple	No destructiva
tiempo-de	Pura	Consulta/accesor (assoc)	No destructiva
secuencia-temporal	Pura	Constructora simple	No destructiva
duracion-ciclo	Pura	Orden superior (mapcar+reduce)	No destructiva
transiciones-validas	Pura	Constructora simple	No destructiva
transicion	Pura	Predicado + condicional	No destructiva
color-en-posicion	Pura	Recursiva de cola	No destructiva
timer	Pura	Composición	No destructiva
formatear-tiempo	Pura	Composición / interop local-time	No destructiva
auditar-cambio	Impura (escribe en pantalla)	Composición	No destructiva
recomendacion-ciclo	Pura	Predicado (cond)	No destructiva
ciclos-por-tiempo	Pura	Composición (floor)	No destructiva
distribucion-horaria	Pura	Orden superior (mapcar)	No destructiva

3. Fase 2 - Autonomía y ecosistema (Quicklisp)

3.1 Quicklisp

Quicklisp es el gestor de paquetes de facto de Common Lisp instalarlo se carga el archivo `quicklisp.lisp` y se ejecuta (`quicklisp-quickstart:install`); luego, cada librería se carga usando (`ql:quickload :nombre`). En el archivo `core.lisp`, la carga se diseñó de forma defensiva. Esto significa que si Quicklisp o la librería no están instalados, el archivo se carga igual y los requerimientos 1-6 siguen funcionando con los tiempos por defecto. Para lograr eso la llamada a la librería se resuelve en tiempo de ejecución mediante `find-package/find-symbol`, evitando un error de lectura si el paquete no existe.

3.2 Librería seleccionada y justificación

La consigna pide integrar una de las dos librerías propuestas. Por lo que Integramos local-time.

local-time - auditoría legible para humanos (Req 3):

La versión base del Requerimiento 3 imprime el instante en formato Unix (epoch), un entero como 1747405800 que es ilegible para un analista forense. Con local-time modificamos la auditoría para que muestre la fecha y hora en formato humano [AAAA-MM-DD HH:MM:SS] (ej. [2025-05-16 14:30:00]). La conversión se encapsula en la función pura formatear-tiempo:

(defun formatear-tiempo (epoch)

(let ((lt (find-package :local-time)))

(if lt

(funcall (find-symbol "FORMAT-TIMESTRING" lt)

nil

(funcall (find-symbol "UNIX-TO-TIMESTAMP" lt) epoch)

:format '(:year 4) #\- (:month 2) #\- (:day 2) #\Space

(:hour 2) #\: (:min 2) #\: (:sec 2))

:timezone (symbol-value (find-symbol "+UTC-ZONE+" lt)))

(format nil "~a" epoch)))) ; fallback: epoch crudo si la lib no está

Usamos unix-to-timestamp para pasar del epoch a un timestamp de local-time y format-timestring con una lista de directivas para componer el string exacto.

El formato se fija en UTC (+utc-zone+) para que la salida sea determinista y no dependa de la zona horaria de la máquina.

Justificación:

Frente a la alternativa (cl-json, que externaliza los tiempos a un .json), local-time resuelve un problema real en la experiencia de uso del programa.

La reconstrucción histórica de estados (el propósito explícito del Req 3) exige fechas legibles, no enteros epoch. Además, el manejo correcto de fechas, time zones y formateo es notoriamente propenso a errores si se hace a mano, por lo que apoyarse en una librería estándar y probada es la decisión de ingeniería más sólida. La

integración es mínimamente invasiva: sólo cambia la presentación final del Req 3; el resto del núcleo funcional, (Req 1, 2, 4, 5, 6) intacto e independiente de la librería.

3.3 Pureza vs. impureza en la integración

La separación efecto/cálculo se mantiene limpia: formatear-tiempo es pura (dado un epoch siempre produce el mismo string en UTC, sin efectos secundarios) y no destructiva; la única función impura es auditar-cambio, porque escribe en la terminal (efecto de salida). Es el patrón funcional clásico de "núcleo puro, cáscara impura": todo el cómputo es puro y el efecto queda aislado en un único borde. La carga de la librería es defensiva (find-package / find-symbol): si local-time o Quicklisp no están disponibles, el archivo igual se carga y formatear-tiempo degrada al epoch crudo (comportamiento base del Req 3), de modo que los Requerimientos 1-6 nunca dejan de funcionar.

4. Bitácora de Depuración

Durante el desarrollo del proyecto fueron apareciendo distintos errores y problemas que tuvimos que analizar y corregir. En esta sección se describen los bugs más importantes que surgieron, explicando qué ocurría, cuál fue la causa del problema y cómo se resolvió.

Para cada caso se deja un espacio destinado a una captura de pantalla del REPL donde pueda observarse el error original o el resultado obtenido después de aplicar la corrección correspondiente.

Bug 1 - La lista citada '(...) no evaluaba las variables

Qué pasaba:

En una de las primeras versiones de la función transición, el resultado no era el esperado. En lugar de devolver el valor que recibía por parámetro, devolvía literalmente el nombre de la variable.

;; INCORRECTO

```
(defun transición (color-actual cambiar-a)
```

```
  '(color-actual "cambiar-a-verde"))
```

```
(transición 'en-rojo 'verde)
```

```
=> (COLOR-ACTUAL "cambiar-a-verde")
```

Por qué pasaba:

Esto ocurrió porque usamos quote (') para crear la lista. Al hacer eso, Lisp no evalúa nada de lo que está dentro de la lista y devuelve exactamente lo que está escrito. Por eso aparecía COLOR-ACTUAL en vez de EN-ROJO.

Cómo lo solucionamos:

Reemplazamos la lista citada por una llamada a list, que sí evalúa los parámetros antes de construir la lista.

```
(list color-actual
```

```
  (format nil "cambiar-a-~(~a~)" cambiar-a))
```

Bug 2 - Problemas al volver a cargar el archivo usando defconstant**Qué pasaba:**

Inicialmente habíamos definido la tabla de transiciones usando una constante global.

```
(defconstant +transiciones+ '((en-rojo . verde) ...))
```

La primera vez funcionaba correctamente, pero cuando volvíamos a cargar el archivo aparecía un error indicando que la constante estaba siendo redefinida.

Por qué pasaba:

Aunque el contenido de las listas era igual, para Lisp cada lista era un objeto distinto en memoria. Entonces interpretaba que estábamos intentando modificar una constante ya existente.

Cómo lo solucionamos:

Decidimos reemplazar las constantes por funciones que devuelven las listas cuando se necesitan. De esta forma evitamos el error y además mantenemos un enfoque más funcional, sin depender de variables globales.

Bug 3 - Desborde de pila por una recursión incorrecta**Qué pasaba:**

Mientras desarrollábamos la función color-en-posición, una versión inicial entraba en recursión infinita y terminaba mostrando un error de pila.

```
Control stack exhausted
```

Por qué pasaba:

La posición no se reducía correctamente en cada llamada recursiva, por lo que nunca

se alcanzaba el caso base. Como consecuencia, la función seguía llamándose a sí misma indefinidamente.

Cómo lo solucionamos:

Agregamos la normalización del tiempo utilizando mod y corregimos la resta de la duración en cada paso de la recursión. Así garantizamos que la posición disminuya y que eventualmente se llegue al caso base.

Bug 4 - Los símbolos aparecían en mayúsculas

Qué pasaba:

Al generar las acciones del sistema obteníamos resultados como:

"cambiar-a-VERDE"

Cuando en realidad queríamos mostrar:

"cambiar-a-verde"

Por qué pasaba:

Common Lisp almacena los símbolos en mayúsculas por defecto. Cuando utilizábamos format, los imprimía exactamente de esa forma.

Cómo lo solucionamos:

Utilizamos la directiva ~(~a~) dentro de format, que convierte automáticamente el texto a minúsculas al momento de mostrarlo.

(format nil "cambiar-a-~(~a~)" 'verde)

Bug 5 - La auditoría mostraba una hora distinta según la computadora

Qué pasaba:

Cuando incorporamos la librería local-time, observamos que la misma fecha y hora aparecía diferente dependiendo de la computadora donde se ejecutaba el programa.

Por qué pasaba:

La librería tomaba automáticamente la zona horaria configurada en cada sistema operativo. Como resultado, la salida variaba entre distintas máquinas.

Cómo lo solucionamos:

Indicamos explícitamente que la conversión debía realizarse en UTC utilizando local-time:+utc-zone+. De esta forma todos los resultados quedaron unificados y los ejemplos eran reproducibles en cualquier entorno.

Bug 6 - El programa no cargaba si la librería local-time no estaba instalada

Qué pasaba:

En equipos donde la librería local-time no estaba instalada, el archivo ni siquiera podía cargarse y aparecía un error relacionado con el paquete LOCAL-TIME.

Por qué pasaba:

Lisp intenta reconocer los paquetes mientras lee el archivo. Si encuentra una referencia a un paquete inexistente, detiene la carga inmediatamente antes de ejecutar cualquier instrucción.

Cómo lo solucionamos:

Implementamos una búsqueda dinámica de los símbolos usando find-package y find-symbol. Así, si la librería no está disponible, el programa continúa funcionando sin interrumpirse y utiliza una alternativa más simple para mostrar la información.

5. Fase 3 - Estudio Comparativo: OCaml

5.1. Presentación del lenguaje

OCaml (Objective Caml) es un lenguaje de programación creado en 1996 por el instituto INRIA de Francia. Se caracteriza por tener tipado estático fuerte e inferencia de tipos, lo que significa que el compilador puede deducir automáticamente los tipos sin necesidad de declararlos en la mayoría de los casos. Además, combina programación funcional con características imperativas y orientadas a objetos.

Entre sus principales características se destacan los tipos algebraicos, el uso de pattern matching, la inmutabilidad por defecto y su compilador eficiente, capaz de generar código nativo de alto rendimiento. Debido a esto, suele decirse informalmente que "si compila, probablemente funcione", ya que muchos errores son detectados antes de ejecutar el programa.

Actualmente, OCaml es utilizado en distintas áreas, entre ellas:

Finanzas y trading de alta frecuencia.

Desarrollo de compiladores y herramientas para lenguajes.

Métodos formales y verificación de software.

Blockchain e infraestructura.

Sistemas y software de bajo nivel.

Algunas empresas y proyectos reconocidos que utilizan o utilizaron OCaml son:

Jane Street, empresa financiera que desarrolla gran parte de sus sistemas en OCaml.

Meta (Facebook), que empleó OCaml para herramientas como Flow, Hack e Infer.

Docker, a través de proyectos relacionados con MirageOS.

Tezos, cuya blockchain está implementada en este lenguaje.

Citrix, mediante el toolstack de Xen.

Coq, asistente de pruebas formales desarrollado en OCaml.

Incluso las primeras versiones del compilador de Rust fueron escritas en este lenguaje.

5.2. Reimplementación de transición y timer

El código completo se encuentra en `comparativa/solucion.ml`.

Se definieron los tipos `estado` y `color`, junto con las funciones `transición` y `timer`. La primera determina la acción correspondiente según el estado actual del semáforo y el color al que se desea cambiar. Por otro lado, `timer` calcula qué color debe mostrar el semáforo según un instante de tiempo determinado dentro del ciclo completo.

Para ello se definieron los tiempos de duración de cada estado:

Rojo: 90 segundos.

Verde: 120 segundos.

Amarillo: 6 segundos.

La suma de estos tiempos da una duración total del ciclo de 216 segundos.

5.3. Pregunta 1 - Inferencia de tipos

En OCaml no fue necesario declarar explícitamente los tipos de las funciones, ya que el compilador los dedujo automáticamente mediante el sistema de inferencia de tipos. Al ejecutar las definiciones en el entorno interactivo (`Ocaml utop`), el sistema muestra las firmas inferidas para cada función.

Por ejemplo, la función `transición` fue interpretada como:

`Estado -> color -> estado * string`

Esto indica que recibe un valor de tipo `estado`, otro de tipo `color` y devuelve una tupla formada por un `estado` y un `string`.

Por su parte, la función `timer` fue inferida como:

int -> color

es decir, recibe un número entero y devuelve un color.

El compilador deduce estos tipos analizando cómo se utilizan los valores dentro del código. En transición, el uso de patrones como `En_rojo o Verde` obliga a que los argumentos pertenezcan a los tipos `estado` y `color`, respectivamente. Además, como todas las ramas devuelven una tupla del mismo tipo, el compilador puede inferir el tipo de retorno.

En el caso de `timer`, el operador `mod` solo trabaja con enteros, por lo que el parámetro debe ser de tipo `int`. A su vez, todas las ramas retornan constructores del tipo `color`.

A diferencia de Common Lisp, que posee tipado dinámico y detecta errores durante la ejecución, OCaml verifica la compatibilidad de tipos en compilación, evitando muchos errores antes de ejecutar el programa.

Aunque es posible agregar anotaciones de tipos para documentar el código o hacerlo más explícito, en general no son necesarias gracias al sistema de inferencia.

5.4. Pregunta 2 - Orientación a expresiones y `match...with`

Una característica importante de OCaml es que prácticamente todo es una expresión que produce un valor. Estructuras como `if...then...else` o `match...with` siempre devuelven un resultado.

Comparándolo con Lisp, la principal diferencia no está tanto en que ambos sean lenguajes orientados a expresiones, sino en las ventajas que ofrece el `match...with` de OCaml.

En primer lugar, permite realizar **pattern matching**, es decir, descomponer estructuras de datos de forma directa. Por ejemplo, en la función `transicion` se evalúan simultáneamente el estado actual y el color deseado mediante una tupla, sin necesidad de escribir múltiples condiciones.

Otra ventaja es la **verificación de exhaustividad**. Si algún caso no fue contemplado, el compilador genera una advertencia indicando que el patrón no es exhaustivo. Esto ayuda a detectar errores antes de ejecutar el programa.

Finalmente, la combinación entre `pattern matching` y tipado estático brinda mayor seguridad. Como los tipos son cerrados, solo pueden utilizarse valores válidos definidos previamente. Un valor inexistente simplemente no compilaría.

El patrón `_` funciona como un caso por defecto, de manera similar al `cond` en Lisp.

En resumen, `match...with` no solo permite seleccionar una rama de ejecución, sino también describir la estructura de los datos y garantizar que todos los casos sean tratados correctamente.

5.5. Conclusión del grupo

El estudio de OCaml después de trabajar con Common Lisp permitió observar diferencias importantes entre ambos enfoques. Mientras que en Lisp muchos errores aparecen durante la ejecución, en OCaml una gran parte de ellos se detectan en compilación gracias al sistema de tipos y al `pattern matching` exhaustivo.

Al principio, adaptarse al tipado estático y a los mensajes del compilador resultó un poco más complejo. Sin embargo, con la práctica se vuelve una herramienta útil para encontrar errores rápidamente.

La inferencia de tipos fue uno de los aspectos más interesantes del lenguaje, ya que brinda la seguridad del tipado estático sin obligar al programador a declarar todos los tipos manualmente.

Además, el uso de `pattern matching` sobre tipos algebraicos hace que el código resulte más claro y expresivo. En general, OCaml ofrece un enfoque diferente al de Lisp, priorizando la seguridad y la detección temprana de errores, algo especialmente valioso en proyectos grandes o críticos.